

From characters to next-token prediction

A fixed context, learned embeddings, a hidden layer, and one distribution over the next token.

Context → embedding lookup → concatenate → hidden ReLU → logits → softmax

1 · Turn a sequence into training pairs

For a window of w tokens, the model estimates

$$p_{\theta}(x_{t+1} \mid x_{t-w+1}, \dots, x_t).$$

The input is the context; the target is the **observed next token**. In the name **aabid**, one training pair is **a a b → i**.

Context

- - -
- - a
- a a
a a b
a b i
b i d

Target

a
a
b
i
d
-

Here - marks the boundary: it pads the initial context and ends generation. Vocabulary: 26 letters
+ - = **27 tokens**.

2 · An ID selects a learned row

A token ID is an index, not a meaningful magnitude. The shared embedding table is

$$E_{\text{tok}} \in \mathbb{R}^{27 \times 2}.$$

Each lookup returns two learned numbers. Rounded rows from the lecture's trained model:

$$e_a \approx [0.63, 0.09]$$

$$e_b \approx [-1.25, -0.15].$$

Both occurrences of a look up the **same row**. Their positions in the concatenated input are different.

These axes are learned coordinates, not letters, token IDs, or fixed human-readable properties.

3 · Preserve the order by concatenating

With $w = 3$ and embedding width $d = 2$,

$$a_0 = [e_a, e_a, e_b] \in \mathbb{R}^{1 \times 6}.$$

This makes **one six-number input**, not six tokens. The first, second, and third slots have their own connections to the hidden layer.

Swapping the tokens swaps input blocks. Summing their embeddings would instead discard this slot information.

4 · Keep the hidden layer in the path

All equations use **row vectors**:

$$a_1 = \text{ReLU}(a_0 W_1 + b_1)$$

$$z = a_1 W_2 + b_2, \quad p = \text{softmax}(z).$$

ReLU acts on each coordinate: $\text{ReLU}(u) = \max(0, u)$. The 32 hidden activations are recomputed for each context. They are not stored parameters.

Without a nonlinear activation, two affine layers collapse into one affine map.

Shape and parameter check

Parameter	Shape	Count
E_{tok}	27×2	54
W_1	6×32	192
b_1	1×32	32
W_2	32×27	864
b_2	1×27	27
Total		1,169

Activation widths: $6 \rightarrow 32 \rightarrow 27$. PyTorch Linear stores each weight transposed relative to this row-vector notation.

5 · Scores become probabilities

A logit z_k is a real-valued score, not a probability. Softmax compares all vocabulary scores:

$$p_k = \frac{\exp(z_k - c)}{\sum_j \exp(z_j - c)}$$

where $c = \max_j z_j$. Every p_k is nonnegative and $\sum_k p_k = 1$. Subtracting the same c keeps the probabilities unchanged and avoids exponential overflow.

Three-token arithmetic example (not the 27-token model): logits $[0, 1, 2]$ give

$$\frac{[1, 2.718, 7.389]}{11.107}$$

$$\approx [0.090, 0.245, 0.665].$$

What changes, and when?

Training changes parameters. Generation reuses them to predict, choose a token, and move the window.

6 · Score the observed next token

If the observed target has index y , its loss is

$$\ell = -\ln p_y = -z_y + \ln \sum_j \exp(z_j).$$

Higher probability for the true token means lower loss. For example, $p_y = 0.1$ gives $\ell \approx 2.303$; $p_y = 0.5$ gives $\ell \approx 0.693$.

Over a batch of B context-target pairs:

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^B \ln p_{b,y_b}.$$

Natural logs measure the loss in **nats**. Perplexity is $\exp(\mathcal{L})$; compare it only under the same tokenization and evaluation setup.

7 · Backprop trains the whole model

For one example, the logit gradient is

$$\frac{\partial \ell}{\partial z_k} = p_k - \mathbb{1}[k = y].$$

The chain rule carries this signal through W_2 , ReLU, W_1 , and the embedding lookups. An SGD step is

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}.$$

θ contains the embedding table and both layers' weights and biases.

For $aab \rightarrow i$, the input uses rows a and b . The two uses of a contribute gradients to the same row. The target i selects the loss; its embedding is not an input lookup for this pair.

The minimal training loop

```
optimizer.zero_grad()
z = model(context_ids)
loss = F.cross_entropy(z, target_ids)
loss.backward()
optimizer.step()
```

Feed raw logits to cross-entropy. Clear old gradients because PyTorch accumulates them. Keep held-out names out of training.

8 · Generate: sample, append, repeat

1. Start with - - - or a chosen prefix.
2. Look up the current three tokens and run the MLP.
3. Choose a token from its 27-way distribution.
4. Append it and keep only the last three tokens.
5. Repeat until - is chosen or a length limit is reached.

Greedy decoding picks the largest probability.

Sampling draws from the distribution; it can choose a non-winning token. Neither requires a parameter update.

Temperature uses $p = \text{softmax}(\frac{z}{\tau})$, $\tau > 0$. Smaller τ sharpens the distribution; larger τ flattens it. It does not retrain the model.

9 · Training is not generation

Training: contexts come from observed text; the true next token is known and supplies the loss. Gradients update parameters.

Generation: the next token is unknown; a chosen output enters the next context. Parameters stay fixed.

Evaluate on held-out text. Split at the name/document level before making overlapping windows, so windows from the same name/document do not leak across the split.

10 · Why move on to attention?

Characters, words, and subwords change the tokens and vocabulary, not the next-token objective.

A fixed window cannot use clues outside it. With hidden width h , concatenating a longer window gives wd inputs and wdh weights in the first layer. Doubling w doubles that weight matrix.

Part 2 asks: can the receiving token read relevant earlier information into a fixed-width update? The representation width stays fixed even when the available context grows.

IDs select rows. Embeddings are learned parameters. Hidden activations depend on the current input.

Source: Part 1 interactive lecture. Architecture and embedding examples follow its trained name model; displayed values are rounded.